

(2) 系统故障的恢复。系统故障的恢复在系统重新启动时自动完成，不需要用户干预。其步骤如下：正向扫描日志文件，找出在故障发生前已经提交的事务，将其事务标识记入重做（Redo）队列。同时找出故障发生时尚未完成的事务，将其事务标识记入撤销（Undo）队列。对撤销队列中的各个事务进行撤销处理：反向扫描日志文件，对每个 Undo 事务的更新操作执行逆操作。对重做队列中的各个事务进行重做处理：正向扫描日志文件，对每个 Redo 事务重新执行日志文件登记的操作。

(3) 介质故障与病毒破坏的恢复。介质故障与病毒破坏的恢复步骤如下：装入最新的数据库后援副本，使数据库恢复到最近一次备份时的一致性状态；从故障点开始反向扫描日志文件，找出已提交事务标识并记入 Redo 队列；从起始点开始正向扫描日志文件，根据 Redo 队列中的记录，重做已完成的任務，将数据库恢复至故障前某一时刻的一致状态。

(4) 有检查点的恢复技术。检查点记录的内容可包括建立检查点时刻所有正在执行的事务清单，以及这些事务最近一个日志记录的地址。采用检查点的恢复步骤如下：从重新开始文件中找到最后一个检查点记录在日志文件中的地址，由该地址在日志文件中找到最后一个检查点记录；由该检查点记录得到检查点建立时所有正在执行的事务清单队列（A）；建立重做队列（R）和撤销队列（U），将 A 队列放入 U 队列中，R 队列为空；从检查点开始正向扫描日志文件，若有新开始的事务 T1，则将 T1 放入 U 队列，若有提交的事务 T2，则将 T2 从 U 队列移到 R 队列，直至日志文件结束；对 U 队列的每个事务执行 Undo 操作，对 R 队列的每个事务执行 Redo 操作。

5.3.5 数据库性能优化

通常，对一个集中式数据库的性能进行优化，可以从硬件升级、数据库设计、索引优化策略和查询优化等方面入手。

1. 硬件升级

要提升数据库的运行速度，最直接的方式就是硬件升级，涉及的硬件包括处理器、内存、磁盘子系统和网络。

处理器的升级主要可以考虑用更高频率的处理器代替现有频率较低的处理器，也可以将单处理器的计算机系统升级为多处理器系统。

内存的升级主要是容量的扩充，当容量扩充以后，数据库服务器可以将更多的数据保存在缓冲区，以减少磁盘 I/O 操作，从而提升数据库的整体性能。

磁盘子系统的性能提升主要体现在两个方面：第一，采用高速磁盘系统替代速度较低的磁盘系统，以减少读盘等待时间，提高响应速度；第二，合理分布磁盘 I/O 到多个设备上，以减少资源竞争，提高并行操作能力。

网络方面的升级主要是对带宽的升级。

2. 数据库设计

在数据库设计阶段就可以着手考虑性能优化问题。对数据库进行设计优化，主要可以从逻辑设计和物理设计两个方面入手。

根据前面的介绍，数据库的规范化程度越高，数据库中的冗余信息就越少。然而，规范化

同时又引入了新的问题，规范化使得关系模式不断被拆解，这样关系模式之间的结构变得越来越复杂，在使用数据时频繁执行连接操作，而连接操作是最耗时的，是数据库性能的制约因素。因此，从某种意义上来说，非规范化（反规范化）可以改善系统的性能。在进行数据库设计时，可以考虑合理增加冗余属性，以提升系统性能，常用的措施如下：

- (1) 将常用的计算属性（例如，总计和最大值等）存储到数据库实体中。
- (2) 重新定义实体，以减少外部属性数据或行数据的开支。
- (3) 将关系进行水平或垂直分割，以提升并行访问度。

数据库逻辑结构的设计固然重要，但物理设计也不可忽视，将数据放在不同的物理位置，有时能对性能提升起到非常关键的作用。例如，可以遵循以下准则：

- (1) 与每个属性相关的数据类型应该反映数据所需的最小存储空间，特别是对于被索引的属性更是如此。例如，能使用 `smallint` 类型的就不要用 `integer` 类型，这样，索引字段可以被更快地读取，而且可以在一个数据页上放置更多的数据行，就相应地减少了 I/O 操作。
- (2) 将一个频繁使用的关系模式分割开，并放在两个单独的智能型磁盘控制器的数据库设备上，这样也可以提高性能。因为有多个磁头在查找，数据分离也能提高性能。
- (3) 将数据库中文本或图像属性的数据存放在一个单独的物理设备上，也可以提高性能。如果使用专用的智能型控制器，就能进一步提高性能。

3. 索引优化策略

索引是提高数据库查询速度的利器，而数据库查询往往是数据库系统中最频繁的操作，因此，索引的建立与选择对数据库性能优化具有重大意义。索引的建立与选择可遵循以下准则：

- (1) 建立索引时，应选用经常作为查询，而不常更新的属性。避免对一个经常被更新的属性建立索引，因为这样会严重影响性能。
- (2) 一个关系上的索引过多会影响 `UPDATE`、`INSERT` 和 `DELETE` 的性能，因为关系一旦进行更新，所有的索引都必须跟着做相应的调整。
- (3) 尽量分析出每个重要查询的使用频率，这样可以找出使用最多的索引，然后可以先对这些索引进行适当的优化。
- (4) 对于数据量非常小的关系不必建立索引，因为对于数据量小的关系而言，关系扫描往往更快，而且消耗的系统资源更少。

4. 查询优化

查询优化也称为应用程序优化，它是数据库性能优化的最后一个环节，同时也是最重要的一个环节。查询语句的构造不当，会使之前的优化功亏一篑。SQL 语句优化的策略很多，例如，建立物化视图或尽可能减少多表查询；以不相干子查询替代相干子查询；只检索需要的属性；用带 `IN` 的条件子句等价替换 `OR` 子句；经常提交（`COMMIT`），以尽早释放锁等。

5.4 数据库设计与建模

数据库设计是指对一个给定的应用环境，提供一个确定最优的数据模型与处理模式的逻辑